AI Engineering from Scratch — инструкция

AI Engineering from Scratch — полная инструкция
Раздел 1. Для тебя (владелец `dechernov-stack/ai-engineering-from-scratch`)
- 1.1. Первый запуск
- 1.2. Окружение (Урок 1 курса — ты это уже прошёл)
- 1.3. Твой рабочий цикл по курсу
- 1.4. Где смотреть материал
- 1.5. Ветки, которые у тебя уже есть

Раздел 2. Для студентов
- 2.1. Форк и клонирование (обязательно форк, прямого доступа у студента нет)
- 2.2. Окружение — то же самое, что в 1.2
- 2.3. Единственное жёсткое правило курса
- 2.4. Как проходить урок
- 2.5. Как подтянуть новые уроки/обновления перевода от преподавателя

Раздел 3. Как жить дальше (сопровождение проекта)
- 3.1. Синхронизация с автором оригинала (rohitg00)
- 3.2. Перевод оставшихся фаз
- 3.3. Если наберётся много студентов (десятки-сотни)
- 3.4. Интеграция с `student` (личный кабинет, авторизация, прогресс, FSRS, чат-тьютор)
- 3.5. Витрина (главная страница сайта)

AI Engineering from Scratch — полная инструкция

Три отдельных раздела: для тебя как владельца репозитория, для студентов, и что делать дальше с проектом. Не смешиваются — каждый читает только свой раздел.

Раздел 1. Для тебя (владелец `dechernov-stack/ai-engineering-from-scratch`)

1.1. Первый запуск

```
cd ~
git clone https://github.com/dechernov-stack/ai-engineering-from-scratch.git
cd ai-engineering-from-scratch
git remote add upstream https://github.com/rohitg00/ai-engineering-from-scratch.git
git remote -v   # origin = твой форк, upstream = оригинал автора
```

У тебя прямой доступ на запись в свой же форк — никакого промежуточного форка-от-форка не нужно, в отличие от студентов (раздел 2).

1.2. Окружение (Урок 1 курса — ты это уже прошёл)

```
# Node через fnm
curl -fsSL https://fnm.vercel.app/install | bash
eval "$(fnm env --shell zsh)"
fnm install 22 && fnm use 22
node -e "console.log('Node', process.version)"

# pnpm
npm install -g pnpm

# Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
rustc --version && cargo --version
```

1.3. Твой рабочий цикл по курсу

```
git checkout -b my-progress
mkdir -p my-work/dechernov-stack
# делаешь код из Build It / Use It каждого урока -> кладёшь в my-work/dechernov-stack/
git add my-work/
```

```
git commit -m "phase 01, lesson 01: vectors from scratch"
git push origin my-progress
```

Папка `my-work/<логин>/` — твои файлы, не трогают контент курса (`phases/`, `site/`, `scripts/`). Это не формальность — именно это правило не даёт твоим апдейтам конфликтовать с чужими правками позже (см. раздел 3).

1.4. Где смотреть материал

- Сайт (частичный перевод): http://72.56.107.117:8181 (`?lang=ru` для интерфейса на русском)
- Локально: `phases/<фаза>/<урок>/docs/en.md` или `ru.md`, если фаза переведена

1.5. Ветки, которые у тебя уже есть

- `main` — код курса, английский канон, сайт
- `translations` — автоматический выход перевода (пишет GitHub Actions, руками не трогать)

Раздел 2. Для студентов

2.1. Форк и клонирование (обязательно форк, прямого доступа у студента нет)

1. На GitHub открой `https://github.com/dechernov-stack/ai-engineering-from-scratch`, нажми **Fork**
2. Клонируй свою копию:

```
git clone https://github.com/ТВОЙ-ЛОГИН/ai-engineering-from-scratch.git
cd ai-engineering-from-scratch
git remote add upstream https://github.com/dechernov-stack/ai-engineering-from-scratch.git
git checkout -b my-progress
```

(`upstream` здесь — форк преподавателя, не оригинал автора курса — студенту оригинал не нужен)

2.2. Окружение — то же самое, что в 1.2

fnm/Node, pnpm, Rust — идентично разделу 1.2, выполняется один раз на своей машине.

2.3. Единственное жёсткое правило курса

Весь свой код и заметки — только в `my-work/ТВОЙ-ЛОГИН/`. Файлы `phases/`, `site/`, `scripts/`, `.github/` не редактировать никогда. Если хочется поменять пример из урока под себя — скопировать файл к себе в `my-work/`, не трогать оригинал.

Это не бюрократия: без этого правила обновления курса от преподавателя будут конфликтовать с твоим кодом при каждой синхронизации.

2.4. Как проходить урок

Структура одна и та же для всех 523 уроков: `Learning Objectives` → `The Problem` → `The Concept` → `Build It` (пишешь сам) → `Use It` (библиотечная версия) → `Ship It` → `Exercises` → `Key Terms` (глоссарий). Код из `Build It`/`Use It` выполнять руками, не просто читать.

```
git add my-work/
git commit -m "lesson 01: vectors from scratch"
git push origin my-progress
```

2.5. Как подтянуть новые уроки/обновления перевода от преподавателя

```
git fetch upstream
git checkout main
git merge upstream/main       # пройдёт без конфликтов, если соблюдалось правило 2.3
git checkout my-progress
git merge main
```

Раздел 3. Как жить дальше (сопровождение проекта)

3.1. Синхронизация с автором оригинала (rohitg00)

Раз в 2-4 недели (автор обновляет редко и большими пачками):

```
git fetch upstream
git log main..upstream/main --oneline   # посмотреть, что нового, до мерджа
git merge upstream/main
```

Конфликт может вылезти только в 4 файлах, где у тебя кастомизация поверх апстрима — проверяй их в первую очередь при каждом мердже: `lib/lesson-translations.js`, `site/ui-i18n.js`, `site/lesson.html`, `site/lang-picker.js`, `languages.json`.

3.2. Перевод оставшихся фаз

18 из 20 фаз ещё не переведены (переведены `00-setup-and-tooling` и `09-reinforcement-learning` на Claude Haiku 4.5, ~$4 за фазу). Запуск по одной фазе:

```
gh workflow run translate --repo dechernov-stack/ai-engineering-from-scratch -f langs=ru -f phase=<НОМЕР-ФАЗЫ>
```

Прогонять по частям, не все 18 разом — так легче контролировать расход и качество на каждом шаге.

3.3. Если наберётся много студентов (десятки-сотни)

- Лимит на пользователя в день/неделю для чата с тьютором (когда появится интеграция со `student`) — иначе расходы на LLM растут линейно с числом студентов
- Чат по умолчанию на дешёвой модели (Haiku), не на Sonnet
- Допуск в закрытую Telegram-группу курса — автоматизировать (список заранее одобренных ID), не подтверждать каждого вручную
- Раз в месяц проверять отставание студенческих форков от твоего:

```
gh api repos/dechernov-stack/ai-engineering-from-scratch/forks --paginate --jq '.[].full_name'
```

3.4. Интеграция с `student` (личный кабинет, авторизация, прогресс, FSRS, чат-тьютор)

ТЗ для Codex уже написано (`ТЗ_AIFS_в_student.md`) — покрывает мультикурсовость, импорт уроков без LLM (структура уже готова: `Key Terms` → глоссарий, секции → контент), особенность курса без живого преподавателя, уровень доступа `P` (всё публично, MIT). Статус: ждёт реализации, есть открытые вопросы к тебе внутри документа (кто формально “лектор” в системе ролей, нужен ли календарь публикации, какой язык канона — en или ru).

3.5. Витрина (главная страница сайта)

`site/index.html` всё ещё ссылается на оригинальный репозиторий автора (кнопка Star, хедер, атрибуция) — это косметика, на контент/перевод не влияет. Решить отдельно: менять на свой форк или оставить как есть, если студенты туда не заходят (используют прямые ссылки на уроки).